



Entrega 4: Tutorial Hola Mundo (Backend)

Facultad de Ingeniería, Universidad Nacional de Colombia

2016701: Ingeniería de Software I

Docente:

Oscar Eduardo Álvarez Rodríguez

Integrantes (Programangos):

Carlos Daniel Albarracín Cruz

Pablo Andrés Pérez Beltrán

David Nickolai Parra Ariza

Jhoan Sebastián Yaya García

Introducción

Este tutorial describe paso a paso cómo construir el backend del proyecto SISA (Sistema de Supervivencia Académica) desde cero. Está dirigido a cualquiera que necesite entender, replicar o extender el entorno de desarrollo backend.

A lo largo del documento se cubre la configuración del entorno, el levantamiento de la base de datos, la estructura del proyecto Django y una prueba funcional completa mediante Postman.

1. Lenguaje y frameworks seleccionados

Para el desarrollo del backend de SISA se ha seleccionado el siguiente stack tecnológico:

- Lenguaje: **Python** (3.12+). Se eligió Python por su legibilidad, amplio ecosistema de librerías y gran adopción en el ámbito académico y profesional.
- Framework: **Django + Django REST Framework (DRF)**. Django proporciona una arquitectura robusta basada en el patrón MVT (Model-View-Template), incluyendo: un potente ORM, sistema de migraciones y panel de administración. DRF extiende Django para construir APIs RESTful de manera rápida y consistente, con soporte integrado.
- Base de datos: **PostgreSQL**. Se utiliza PostgreSQL como motor de base de datos relacional por su confiabilidad, soporte de transacciones ACID, tipos de datos avanzados (JSONB, arrays) y excelente rendimiento en aplicaciones web.

2. Librerías ORM

El ORM seleccionado es el **ORM propio de Django** (`django.db.models`). Django ORM es un mapeador objeto-relacional maduro y completo que permite:

- Definir modelos de datos como clases Python, que se traducen automáticamente a tablas en la base de datos.
- Realizar consultas a la base de datos mediante una API intuitiva y expresiva (QuerySet API), sin necesidad de escribir SQL manualmente.
- Trabajar con PostgreSQL aprovechando características como JSONB e índices avanzados.

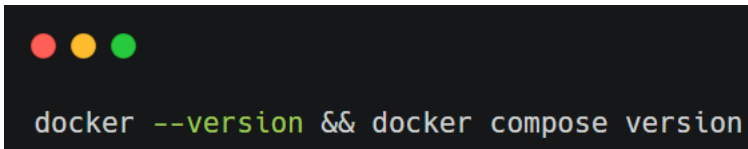
No se requieren librerías ORM adicionales: Django ORM es suficiente y está completamente integrado con el framework, lo que simplifica la configuración y el mantenimiento del proyecto.

3. Configuración y levantamiento de la base de datos

La base de datos utilizada es **PostgreSQL 16**, levantada mediante **Docker** para garantizar un entorno reproducible e independiente del sistema operativo.

Antes de iniciar el proyecto, asegúrate de tener instalados: **Docker** y **Docker Compose**.

-Ejecuta:

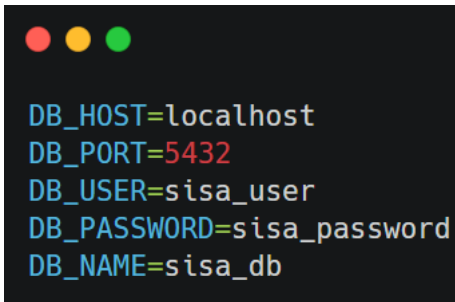


```
docker --version && docker compose version
```

Si ambos comandos retornan una versión, estás listo. De lo contrario, instálalos desde <https://docs.docker.com/get-docker>.

Paso 1: Configura las variables de entorno

Crea un archivo **.env** en la raíz del proyecto backend con el siguiente contenido:



```
DB_HOST=localhost  
DB_PORT=5432  
DB_USER=sis_a_user  
DB_PASSWORD=sis_a_password  
DB_NAME=sis_a_db
```

Estas variables serán consumidas tanto por Docker al levantar el contenedor, como por Django al establecer la conexión con la base de datos.

Paso 2: Crea el archivo docker-compose.yml

Crea un archivo llamado **docker-compose.yml** en la raíz del proyecto backend. Este archivo define el servicio **sis_a_db**: levanta un contenedor PostgreSQL, expone el puerto configurado en **.env**, y monta dos volúmenes: uno para garantizar la persistencia de los datos entre reinicios y otro para ejecutar automáticamente el esquema la primera vez que se inicializa el contenedor.

```

services:
  sisa_db:
    image: postgres:16
    container_name: sisa_postgres
    ports:
      - "${DB_PORT}:5432"
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql
volumes:
  postgres_data:

```

Paso 3: Crea el esquema inicial de la base de datos

Crea un archivo **scripts/init.sql**. Este archivo será ejecutado automáticamente por Docker al inicializar el contenedor, y debe contener la definición de todas las tablas del proyecto. A modo de ejemplo, las dos primeras tablas que se deben definir son **role** y **user**, que establecen la base del sistema de usuarios de SISA:

```

CREATE TABLE role (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255)
);

CREATE TABLE "user" (
  id SERIAL PRIMARY KEY,
  full_name VARCHAR(255),
  email VARCHAR(255) UNIQUE,
  password_hash VARCHAR(255),
  major VARCHAR(255),
  current_semester INT,
  reputation_points INT DEFAULT 0,
  is_active BOOLEAN DEFAULT true,
  role_id INT,
  created_at TIMESTAMP,
  updated_at TIMESTAMP,

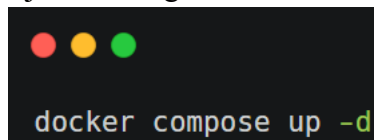
  CONSTRAINT fk_user_role
    FOREIGN KEY (role_id) REFERENCES role(id)
);|

```

El esquema completo se encuentra en **scripts/init.sql** e incluye tablas para edificios, consejos, trámites, noticias, logros y reportes.

Paso 4: Levanta el contenedor

Ejecuta el siguiente comando desde la raíz del proyecto backend:

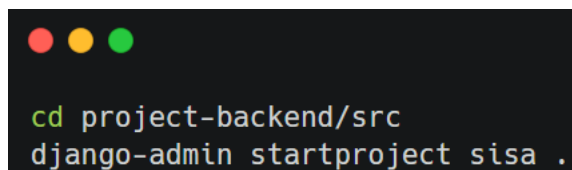


```
docker compose up -d
```

Docker descargará la imagen de PostgreSQL si no se encuentra localmente, creará el contenedor **sisa_postgres**, ejecutará el **init.sql** automáticamente y dejará la base de datos lista para recibir conexiones en **localhost:5432**.

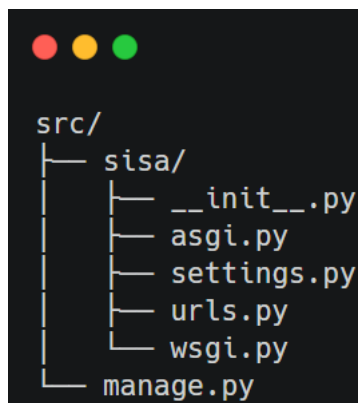
Paso 5: Inicializa el proyecto Django

Dentro de la carpeta **src**, inicializa el proyecto Django con el siguiente comando:



```
cd project-backend/src
django-admin startproject sisa .
```

El punto al final le indica a Django que cree el proyecto en el directorio actual en lugar de crear una carpeta adicional. Esto generará la siguiente estructura:



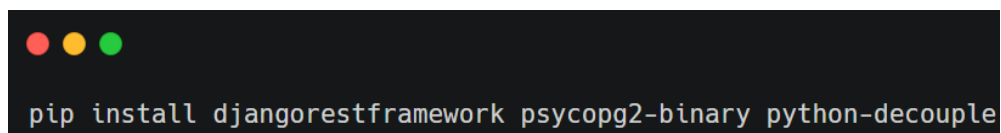
```
src/
├── sisa/
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── manage.py
```

A partir de aquí, **settings.py** es el archivo central de configuración del proyecto y **manage.py** es la herramienta de línea de comandos para interactuar con Django.

Paso 6: Configura Django para conectarse a PostgreSQL

Por defecto, Django genera el **settings.py** con SQLite como base de datos. Necesitas reemplazar esa configuración por PostgreSQL y conectarla a las variables del **.env**.

Primero, instala las dependencias necesarias:



```
pip install django restframework psycopg2-binary python-decouple
```

- **django-rest-framework:** habilita la API REST en Django.
- **psycopg2-binary:** adaptador que permite a Django conectarse a PostgreSQL.
- **python-decouple:** permite leer variables de entorno desde el archivo **.env**.

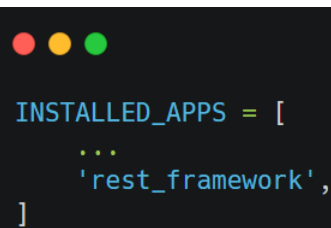
Luego, aplica los siguientes tres cambios en **src/sisa/settings.py**:

1. Agrega la importación de **decouple** al inicio del archivo:



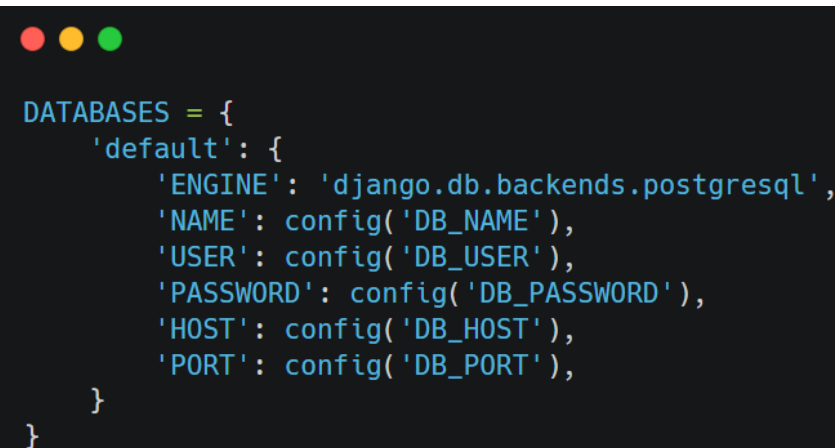
```
from decouple import config
```

2. Agrega **rest_framework** a **INSTALLED_APPS**:



```
INSTALLED_APPS = [
    ...
    'rest_framework',
]
```

3. Reemplaza el bloque **DATABASES** por:



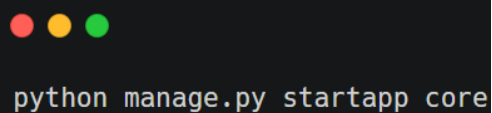
```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': config('DB_NAME'),
        'USER': config('DB_USER'),
        'PASSWORD': config('DB_PASSWORD'),
        'HOST': config('DB_HOST'),
        'PORT': config('DB_PORT'),
    }
}
```

Con esto, Django leerá automáticamente las credenciales del **.env** y se conectará al contenedor PostgreSQL que fue levantado en el paso anterior.

4. Hola Mundo – Implementación y prueba del backend

Paso 7: Crea la app Django core

Dentro de **src/**, ejecuta el siguiente comando para crear la app principal del proyecto:



```
python manage.py startapp core
```

Esto genera la carpeta **core/** con la siguiente estructura base:

```

core/
├── migrations/
│   └── __init__.py
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
└── views.py

```

Luego, registra la app en **settings.py** agregándola a **INSTALLED_APPS**:

```

INSTALLED_APPS = [
    ...
    'rest_framework',
    'core',
]

```

Con esto Django reconoce la app y podrá detectar sus modelos, migraciones y configuraciones automáticamente.

Paso 8: Crea la estructura de capas

Dentro de **core/**, crea las carpetas que representan cada capa de la arquitectura y agrega un **__init__.py** en cada una para que Python las reconozca como módulos:

```

mkdir domain infra services controllers serializers
touch domain/__init__.py infra/__init__.py services/__init__.py controllers/__init__.py
serializers/__init__.py

```

La estructura resultante es:

```

core/
├── controllers/  ← recibe y responde peticiones HTTP
├── domain/      ← define las entidades del negocio
├── infra/       ← accede a la base de datos via ORM
├── serializers/ ← convierte entidades a JSON
├── services/    ← contiene la lógica de negocio
├── migrations/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── tests.py
└── views.py

```

Cada carpeta tiene una responsabilidad única y se comunica únicamente con las capas adyacentes, manteniendo el código organizado y fácil de extender.

Paso 9: Define el modelo Role en domain/

Crea el archivo `core/domain/role.py` con el siguiente contenido:

```
from django.db import models

class Role(models.Model):
    name = models.CharField(max_length=255)

    class Meta:
        db_table = 'role'
        managed = False

    def __str__(self):
        return self.name
```

Este archivo representa la entidad **Role** como una clase Python. Dos detalles importantes:

- `db_table = 'role'` le indica a Django el nombre exacto de la tabla en PostgreSQL a la que debe mapear esta clase.
- `managed = False` le dice a Django que esa tabla ya existe y que no debe intentar crearla ni modificarla al correr migraciones, ya que el esquema es gestionado por `init.sql`.

Paso 10: Crea el serializer en serializers/

Crea el archivo `core/serializers/role_serializer.py` con el siguiente contenido:

```
from rest_framework import serializers
from core.domain.role import Role

class RoleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Role
        fields = ['id', 'name']
```

El serializer es el responsable de convertir instancias del modelo **Role** a formato JSON para ser enviadas como respuesta HTTP. Al heredar de **ModelSerializer**, Django REST

Framework genera automáticamente la serialización basándose en los campos definidos en **fields**.

Paso 11: Crea el repositorio en infra/

Crea el archivo **core/infra/role_repository.py** con el siguiente contenido:

```
from core.domain.role import Role

class RoleRepository:
    def get_all(self):
        return Role.objects.all()
```

El repositorio es la única capa que interactúa directamente con la base de datos.

Role.objects.all() es una consulta del ORM de Django que equivale a **SELECT * FROM role**. Ninguna otra capa debe acceder a la BD directamente. Si en el futuro se cambia el ORM o la fuente de datos, solo este archivo necesita modificarse.

Paso 12: Crea el servicio en services/

Crea el archivo **core/services/role_service.py** con el siguiente contenido:

```
from core.infra.role_repository import RoleRepository

class RoleService:
    def __init__(self):
        self.repository = RoleRepository()

    def get_all_roles(self):
        return self.repository.get_all()
```

El servicio actúa como intermediario entre el repositorio y el controller. Su responsabilidad es contener la lógica de negocio. En este ejemplo es simple, pero en un escenario real aquí se aplicarían validaciones, filtros o transformaciones antes de devolver los datos. Nótese que el servicio no sabe nada de HTTP ni de JSON, solo trabaja con objetos **Role**.

Paso 13: Crea el controller en controllers/

Crea el archivo **core/controllers/role_controller.py** con el siguiente contenido:

```

from rest_framework.views import APIView
from rest_framework.response import Response
from core.services.role_service import RoleService
from core.serializers.role_serializer import RoleSerializer

class RoleController(APIView):
    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.service = RoleService()

    def get(self, request):
        roles = self.service.get_all_roles()
        serializer = RoleSerializer(roles, many=True)
        return Response(serializer.data)

```

El controller es el punto de entrada de cada petición HTTP. Recibe el **request**, llama al servicio para obtener los datos, se los pasa al serializer para convertirlos a JSON y retorna la respuesta. Es importante que no contenga lógica de negocio ni consultas a la BD, ya que esas responsabilidades pertenecen a las capas inferiores.

Paso 14: Registra las URLs

Crea el archivo **core/urls.py** con el siguiente contenido:

```

from django.urls import path
from core.controllers.role_controller import RoleController

urlpatterns = [
    path('roles/', RoleController.as_view(), name='roles'),
]

```

Luego, modifica **sisu/urls.py** para incluir las URLs de **core**:

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('core.urls')),
]

```

El primer archivo define el endpoint **roles/** y lo conecta al controller. El segundo lo registra bajo el prefijo **api/**, de modo que el endpoint final quedará disponible en <http://localhost:8000/api/roles/>.

Paso 15: Levanta el servidor y prueba en Postman

Con todo configurado, levanta el servidor de desarrollo de Django:

```
cd project-backend/src
python manage.py runserver
```

Se debería ver en consola “Starting development server at <http://127.0.0.1:8000/>”

Para verificar que el endpoint devuelve datos reales, inserta los roles iniciales directamente en la BD:

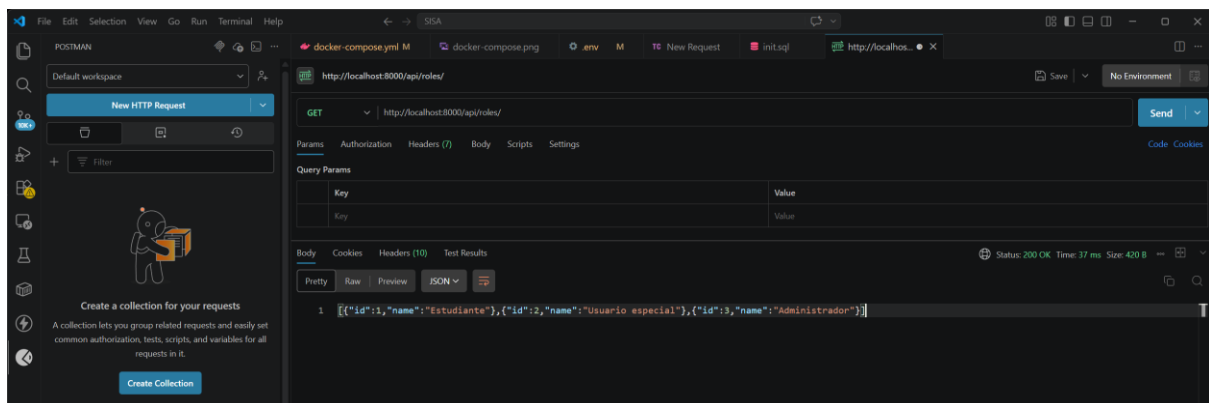
```
docker exec -it sisa_postgres psql -U sisa_user -d sisa_db -c "INSERT INTO role (name) VALUES ('Estudiante'), ('Usuario especial'), ('Administrador');"
```

Estos tres roles corresponden a los perfiles definidos en el sistema SISA.

Abre Postman y realiza la siguiente petición:

- **Método:** GET
- **URL:** <http://localhost:8000/api/roles/>

La respuesta esperada es un **200 OK** con los roles registrados en la BD:



Nota:

Para automatizar todos los pasos descritos en este tutorial, el proyecto incluye un script **setup.sh** en la raíz del repositorio que se encarga de instalar dependencias, levantar el contenedor de la base de datos y preparar el entorno por primera vez. Para usuarios Windows, existe también una versión equivalente **setup.bat**.